Statistical and Computational Models

Lecture 16: Trees, random forests, and boosting

Daniel de Kadt

Department of Government, Cornell University · Fall 2026

Roadmap

Estimating $\pi(\boldsymbol{x})$

Trees

Scoring a classifier

Many trees

Monitoring polling stations

Suppose an election is approaching. An observer mission has $k = 500$ monitors, who can be assigned to 5,000 possible polling stations.

Assume a monitor at a station deters malfeasance there and that malfeasance is never reallocated. Where should the mission assign monitors?

Find the k stations that are most likely to have irregularities, and assign observers accordingly.

This is a **prediction** problem: we need $\mathbb{P}(Y_{i,t+1} = 1 \mid \boldsymbol{X}_{i,t+1} = \boldsymbol{x}_{i,t+1})$ for every station i, where $t + 1$ is this election and t the last one.

Our data: an audit of last election

After the last election, t, an audit checked **every** station's tally sheets.

$Y_{it} = 1$ if the audit detected an irregularity, found in 26% of stations.

y	urban	no_agent	flagged	region	inc_share	turnout
0	0	1	0	Coast	0.64	0.76
0	1	1	3	Coast	0.47	0.56
0	1	0	1	South	0.58	0.90
0	1	1	1	Coast	0.40	0.66

Six features $\boldsymbol{x}_{it}$ are also available prior to election day: `urban`, `no_agent` (no opposition agent registered), `flagged` (of the last eight elections, how many flagged this station), `region`, and the previous election's `inc_share` (the incumbent's share) and `turnout` (so for t, those of $t-1$).

These 5,000 stations, $(\boldsymbol{x}_{it}, y_{it})$, are our **training** data. Our **test** data are this election's $(\boldsymbol{x}_{i,t+1}, Y_{i,t+1})$, with $Y_{i,t+1}$ as yet unrealized.

The target and the decision

The estimation target:

$$\pi(\boldsymbol{x}) = \mathbb{P}(Y_{it} = 1 \mid \boldsymbol{X}_{it} = \boldsymbol{x})$$

Read: the chance the audit detects an irregularity at a station with characteristics $\boldsymbol{X}_{it} = \boldsymbol{x}$.

Assume π is the **same** function at t and $t+1$: we estimate it from election t, then apply it to the $\boldsymbol{x}_{i,t+1}$.

The decision: send the k monitors to the k stations with the highest $\hat{\pi}$.

So what matters is not just $\hat{\pi}$ but also its **order**: the stations with the highest $\hat{\pi}$ should be the ones most likely to have an irregularity. If every $\hat{\pi}$ were overestimated by the same amount, the same 500 stations would still come out on top.

What about logit?

We have the outcome and six features. Suppose we decide to model $\pi(\boldsymbol{x})$ as a logit. Which logit?

1. Each feature enters once, as is (`region` as five dummies): 11 parameters.
2. `flagged` as categories, not a count: 18.
3. A missing agent matters differently in cities, `urban` $\times$ `no_agent`: 12.
4. Curves in `inc_share` and `turnout` (add squares): 13.
5. Every pair of features interacts: 107.
6. Every three features interact: 294.

All six are specifications someone might fit. Each is a **parametric** guess about how the features relate to $\pi(\boldsymbol{x})$.

Why not just take option 6? It **overfits**: 294 parameters, estimated from 1,288 detected irregularities, will fit last election's noise as well as its signal.

Roadmap

Estimating $\pi(\boldsymbol{x})$

Trees

Scoring a classifier

Many trees

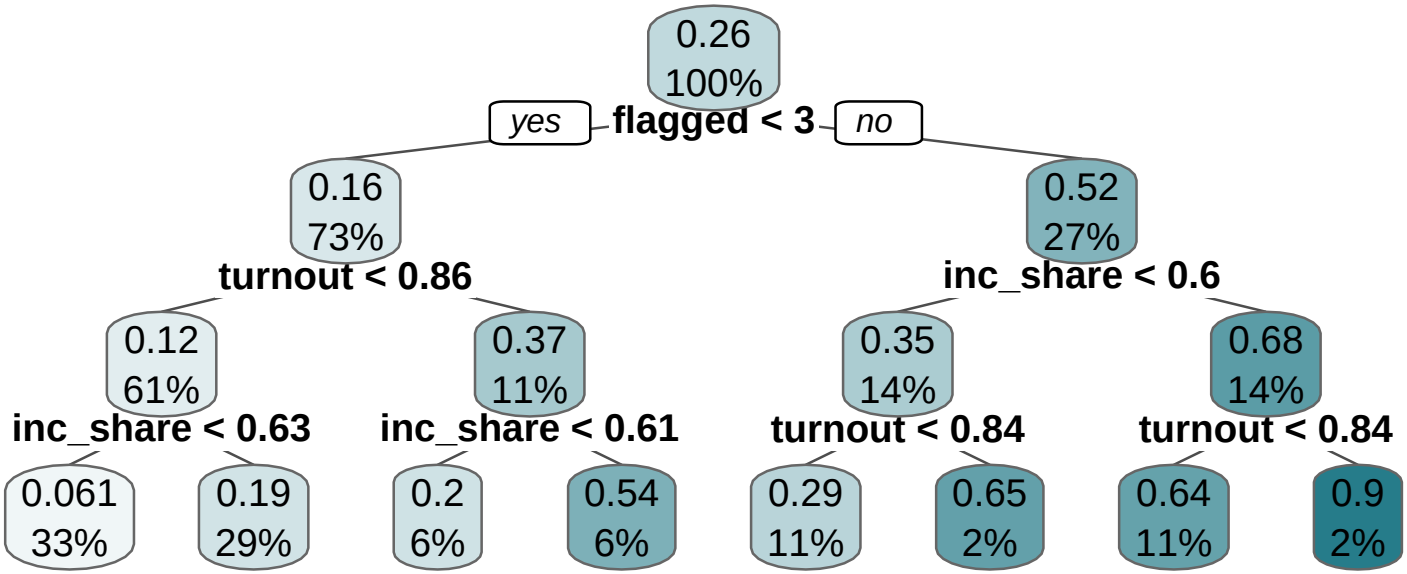
The tree algorithm

A **tree** sorts the stations into groups by asking yes/no questions about their features. Start with all the stations in one group, then:

1. Split the group in two:
    a. Propose possible splits that leave two groups: try every feature, and every cut point within every feature.
    b. Keep the split that is most discriminating: where the two proposed groups differ most in their share with a detected irregularity, $\hat{p}_m = \frac{1}{n_m} \sum_{i \in m} y_i$, weighted by their size.
2. Repeat step 1 inside each new group.
3. Stop when the groups get too small. Each final group is a **leaf**, and its share is $\hat{\pi}$ for every station in it.

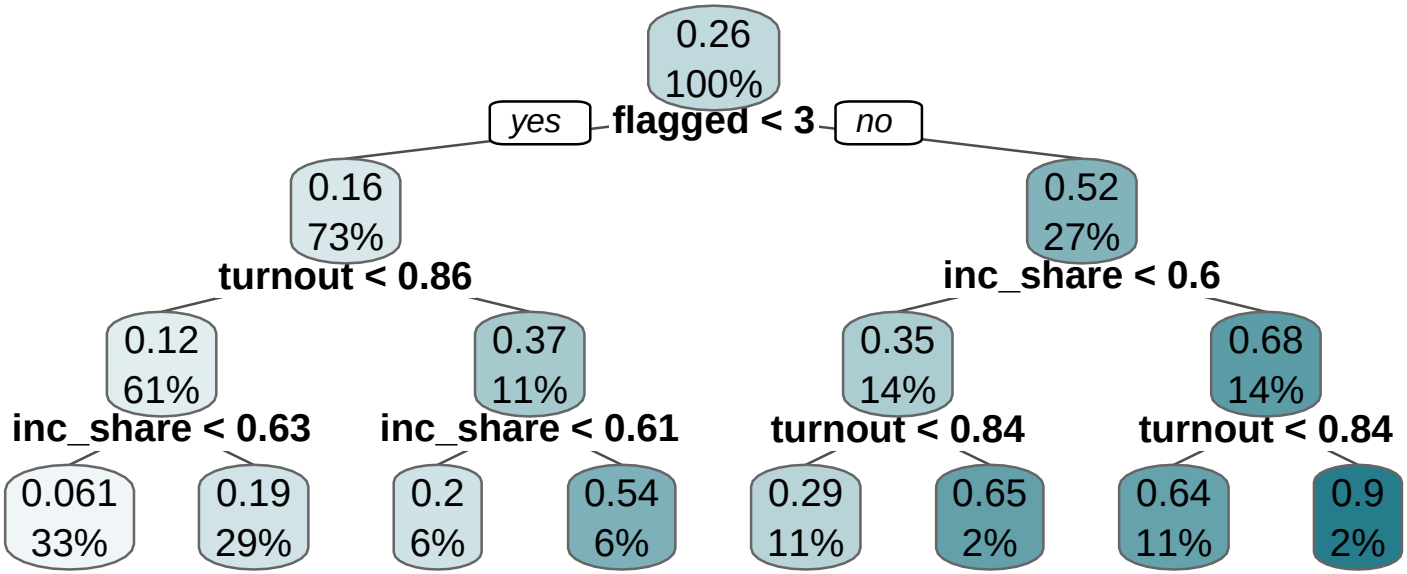
Steps 1b and 3 each involve a researcher choice, and we'll come back to both shortly.

Growing a tree



Stop after three questions. Each final group is a **leaf**, and its share is $\hat{\pi}$ for every station in it.

A tree



Each box: $\hat{\pi}$ in that group, then its share of stations. $\hat{\pi}$ follows a **step function**.

After the first split, each additional split asks a **conditional question** – in regression parlance, an **interaction**.

Choice 1: how to choose a split

In step 1 the algorithm proposes numerous splits. To find the most discriminating, we need a **measure of dissimilarity**.

The **Gini index** is a common default:

$$G = \sum_{m=1}^{2} \underbrace{\frac{n_m}{n}}_{\text{group } m\text{'s share}} \underbrace{\hat{p}_m(1 - \hat{p}_m)}_{\text{how mixed group } m \text{ is}}$$

$m = 1, 2$ are the proposed groups: n_m of the n stations go to group m, and $\hat{p}_m$ is its share with a detected irregularity.

Read: $\hat{p}_m(1 - \hat{p}_m)$ is 0 if every station in group m is alike, largest at half and half. G is its size-weighted average over the two groups: lower G, purer groups.

Note: entropy (based on deviance) gives very similar trees. For a 0/1 outcome, G is the residual sum of squares over n: Gini and **least squares** choose the same splits.

Example: choosing the first split

With all observations in a single group, our six features discriminate to differing degrees:

Feature	Best proposed split	G
`urban`	urban or not	0.188
`no_agent`	agent or not	0.188
`flagged`	3 or more, or fewer	0.166
`region`	North and South or the rest	0.188
`inc_share`	at 0.61	0.181
`turnout`	at 0.86	0.182

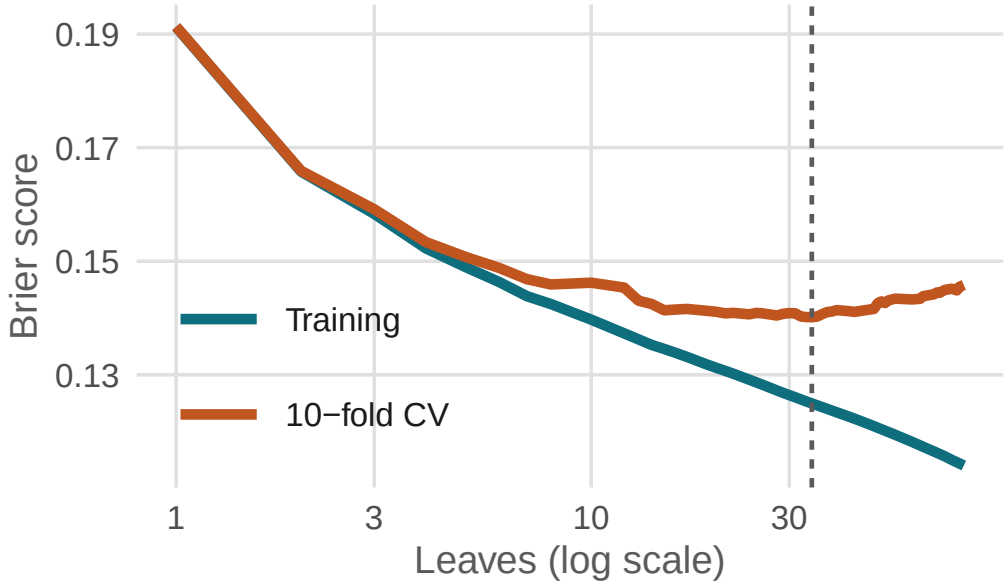
Before any split, all stations are one group: $G = 0.258 \times 0.742 = 0.191$. Every proposed split lowers it, but `flagged` lowers it most.

Step 2 repeats this inside each group and never revisits a split: the search is **greedy**.

Choice 2: when to stop growing

If all we do is propose splits and keep the most discriminating, the tree grows until every leaf is pure: all units have the same outcome, and leaves have very few units. This risks **overfitting**.

When to stop is thus a **tuning parameter**: a setting we choose rather than estimate, which governs how flexible the tree is.



We measure error with the **Brier score**:

$$\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{\pi}_i)^2$$

the mean squared error of $\hat{\pi}$ against the 0/1 outcome.

We tune using CV, not the training curve: grow a big tree, then **prune** it back to the size with the lowest CV error.

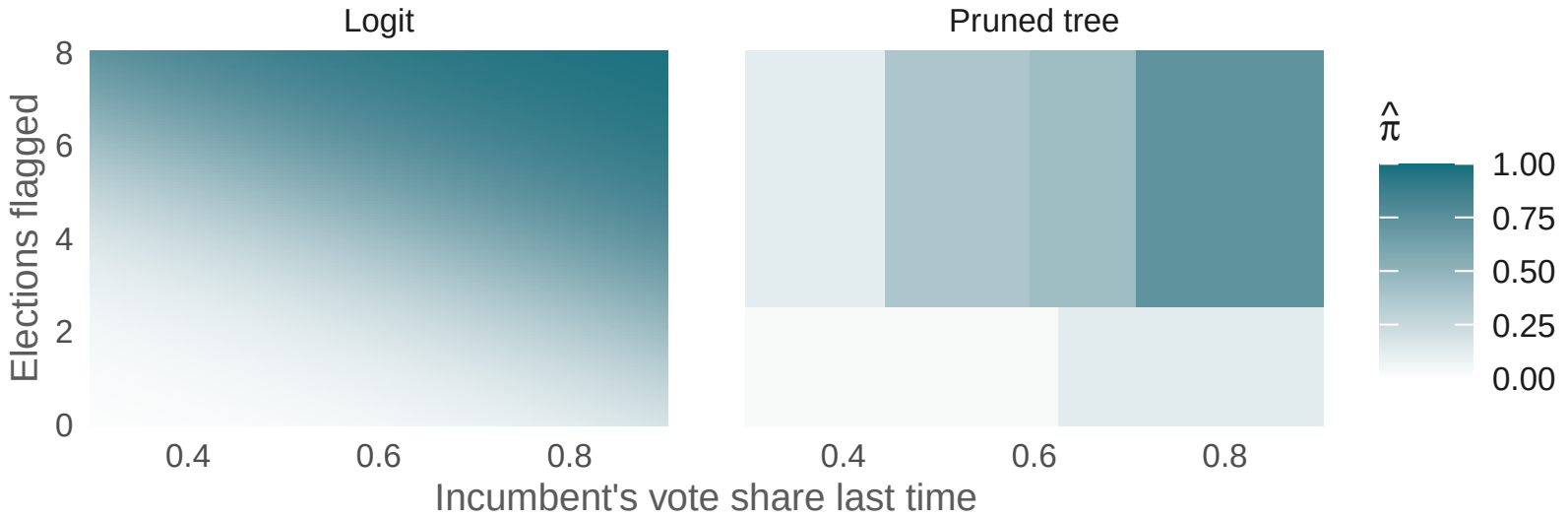
Logit or a tree?

Logit

- We specify a model
- Interactions and jumps only if we add them
- $\hat{\pi}$ changes smoothly, at a pace set by each $\hat{\beta}$

Tree

- Data-driven splits
- Interactions and jumps arise from the splits, not from our specification
- $\hat{\pi}$ moves in steps: every station in a leaf gets the same $\hat{\pi}$



Roadmap

Estimating $\pi(\boldsymbol{x})$

Trees

Scoring a classifier

Many trees

How should we score a classifier?

Here i runs over this election's stations: y_i is $y_{i,t+1}$.

Suppose we label each station by cutting $\hat{\pi}$ at a threshold q: $\hat{y}_i = \mathbf{1}\{\hat{\pi}_i \geq q\}$.

Comparing labels with this election's (simulated) outcomes gives a **confusion matrix**. The logit at $q = 0.5$:

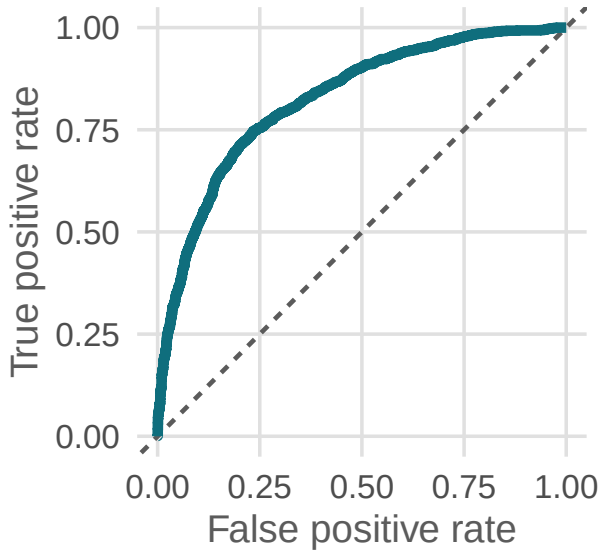
	$y_i = 0$ (none detected)	$y_i = 1$ (detected irregularity)
$\hat{y}_i = 0$	3446 true negatives	748 false negatives
$\hat{y}_i = 1$	246 false positives	560 true positives

True positive rate = $\frac{\text{TP}}{\text{TP+FN}} = 0.43$: the share of irregular stations labelled irregular. At $q = 0.5$ we miss most of them.

False positive rate = $\frac{\text{FP}}{\text{FP+TN}} = 0.07$: the share of clean stations labelled irregular.

Accuracy = $\frac{\text{TP+TN}}{n} = 0.80$: the share of stations labelled correctly. Is that good? A classifier that just assigns every station the "clean" label would score 0.74.

The ROC curve



For a given model of $\hat{\pi}$, each threshold q gives one confusion matrix, and so one (FPR, TPR) pair. The **ROC curve** plots that pair for every q, from $q = 1$ (no station labelled irregular, bottom left) to $q = 0$ (every station, top right). The diagonal is a coin-flip model: $\hat{\pi}$ assigned at random, unrelated to the features.

Lowering q only adds stations in the order the model ranks them, so the curve depends only on the ranking. A better ranking bows toward the top left: it reaches irregular stations (up) before clean ones (right).

Scoring the order

To properly compare models we need a metric for the quality of a proposed **ordering**. The area under the ROC curve:

$$\text{AUC} = \text{area under the curve} = \mathbb{P}\left(\hat{\pi}_i > \hat{\pi}_j \mid y_i = 1,\ y_j = 0\right)$$

Read: take one station where the audit detected an irregularity and one where it did not. The area equals the chance the model ranks the first one higher (ties count half).

A coin-flip model scores 0.5 on average, a perfect ranking 1. The proposed logit scores 0.825 on this election's stations, while the proposed tree scores 0.821.

Roadmap

Estimating $\pi(\boldsymbol{x})$

Trees

Scoring a classifier

Many trees

The bootstrap

To grow many trees we need many training sets, and we have one. So we draw n stations **with replacement** from it. With ten:

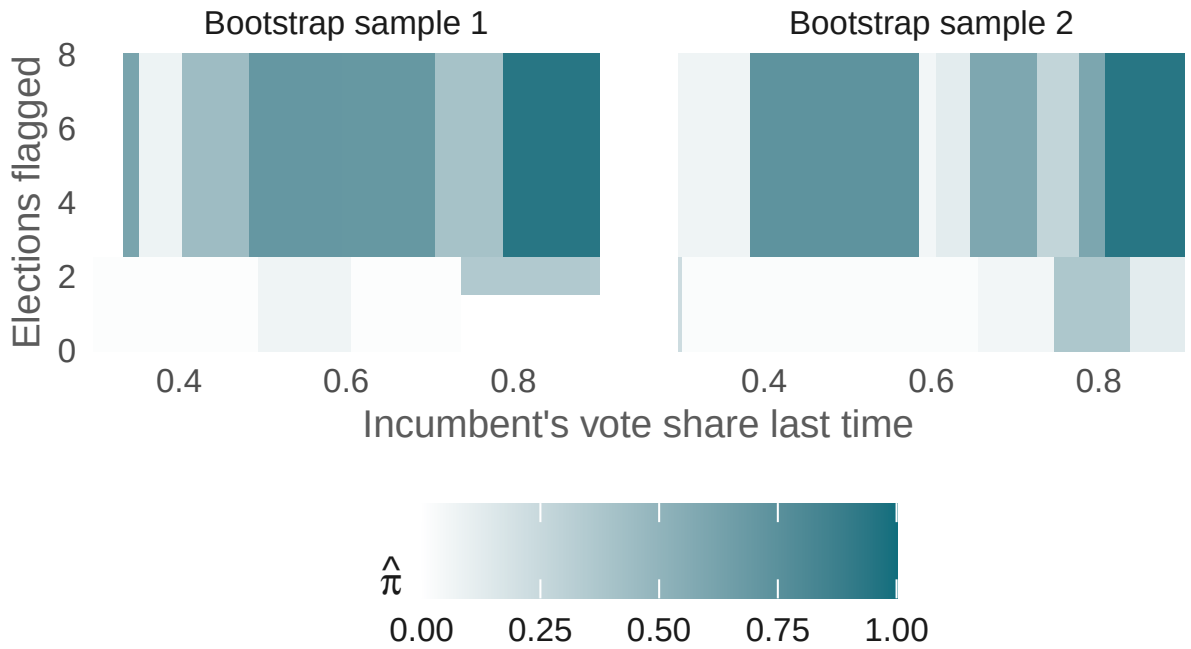
$$\{1, 2, \ldots, 10\} \quad \rightsquigarrow \quad \{1, 2, 4, 6, 6, 8, 9, 9, 9, 10\}$$

$$\mathbb{P}(\text{station } i \text{ left out}) = \left(1 - \frac{1}{n}\right)^n \approx e^{-1} \approx 0.37$$

Read: each bootstrap sample leaves out about a third of the stations. These are its **out-of-bag** stations: a built-in test set for the tree grown on that sample, so we get a CV-like error estimate without refitting.

Each bootstrap sample is a "new" training set, **resampled** from the one we have. Here we use the bootstrap to build models.

One tree is noisy



Two deep trees, grown on two bootstrap samples of last election's stations.

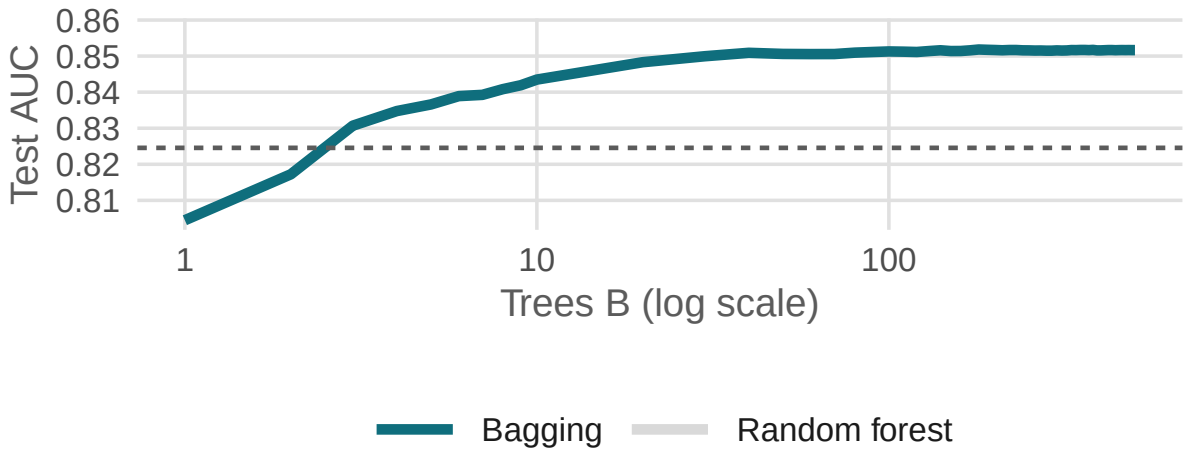
The two trees split differently and give different $\hat{\pi}$. A deep tree has low **bias** but high **variance**.

What if we average many of them?

Bagging

Bagging (bootstrap aggregation):

1. Draw B bootstrap samples.
2. Grow a big tree on each.
3. Average the B estimates: $\hat{\pi}_{\text{bag}}(\boldsymbol{x}) = \frac{1}{B}\sum_{b=1}^{B}\hat{\pi}^{*b}(\boldsymbol{x})$



Averaging cuts the variance, but the gain levels off quickly.

Averaging correlated trees

Each $\hat{\pi}^{*b}(\boldsymbol{x})$ has variance σ^2, and any two are correlated, ρ:

$$\mathrm{Var}\left[\frac{1}{B}\sum_{b=1}^{B}\hat{\pi}^{*b}(\boldsymbol{x})\right] = \rho\,\sigma^2 + \frac{1-\rho}{B}\,\sigma^2$$

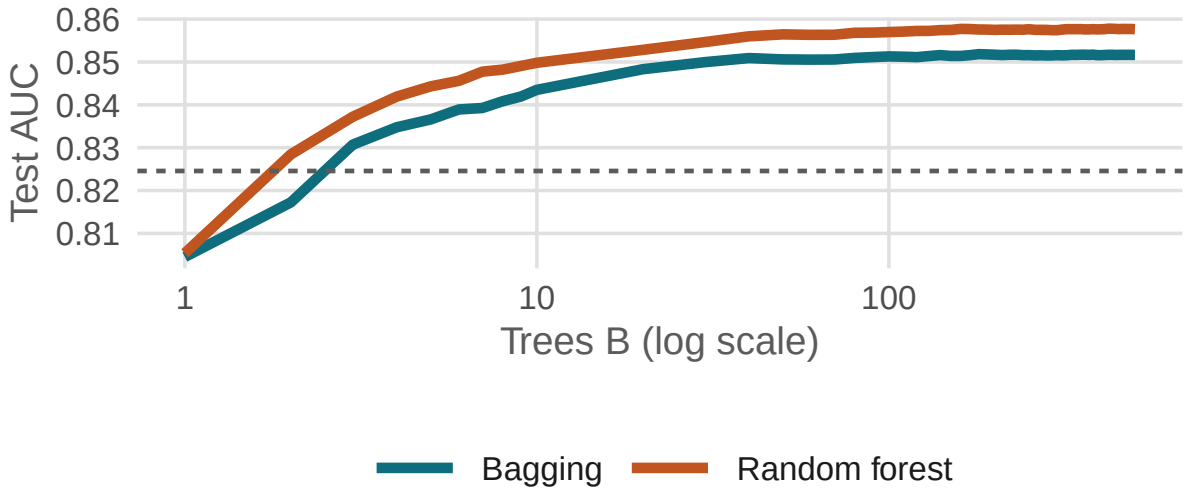
Read: more trees shrink only the second term. The first stays, however large B gets.

Bagged trees are **highly correlated**: grown on overlapping samples, they all split first on the same strong features.

To lower the variance further, we have to lower ρ.

Random forests

A **random forest**: at each split, we consider only m of the p features, drawn at random (here $m = 2$ of 6).



The trees are less alike, so ρ is smaller. With $m = p$ we are back to bagging. We choose m by CV, or on the out-of-bag stations.

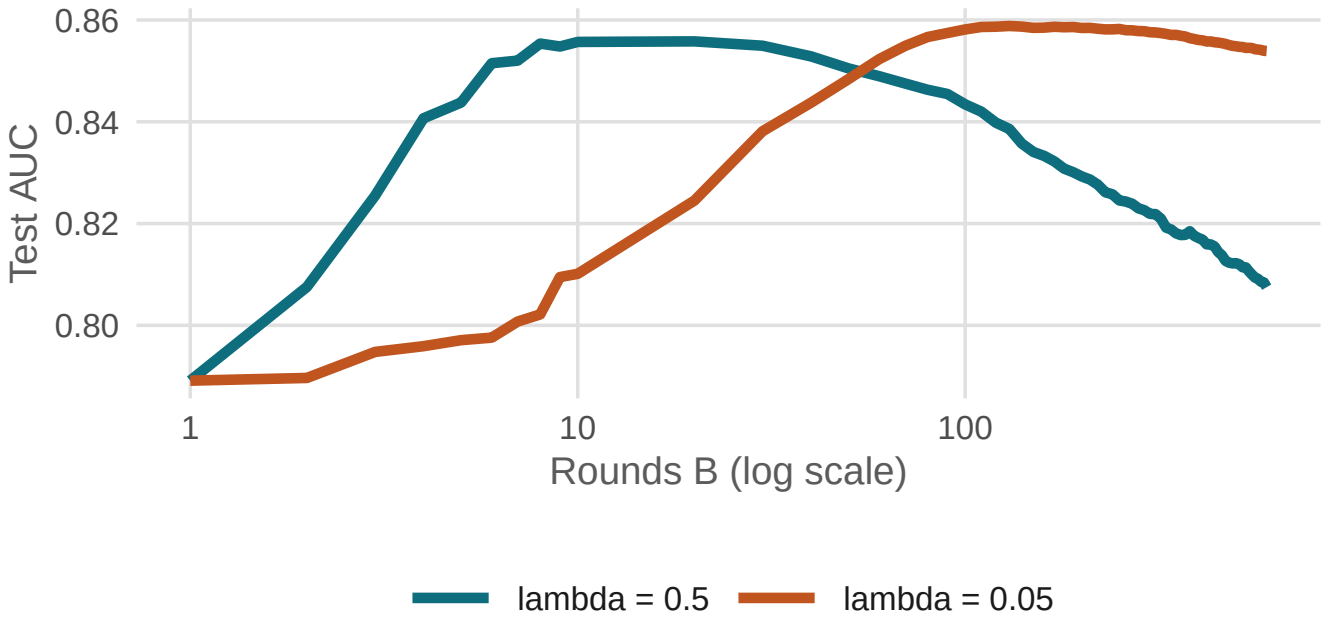
Boosting

A forest attacks one tree's **variance**: grow many big trees independently, then average them. **Boosting** attacks **bias** instead: grow small trees one after another, each fixing the mistakes of the ones before.

1. Start every station at the overall share: $\hat{\pi}_i = \bar{y} = 0.26$.
2. Find the mistakes: the residuals $y_i - \hat{\pi}_i$.
3. Fit a small tree (depth d, here 3) to the residuals: which kinds of station are we getting wrong, and in which direction?
4. Nudge every $\hat{\pi}_i$ a small step λ in the direction that tree points.
5. Repeat steps 2–4 B times.

The result is a sum of many small trees. (The steps are taken on the log-odds scale, so $\hat{\pi}$ stays between 0 and 1.)

Tuning boosting



Big steps ($\lambda = 0.5$) learn fast, then start fitting noise: test AUC falls as rounds are added.

Small steps ($\lambda = 0.05$) learn slowly and end higher. The number of rounds B, the step size λ and the tree depth d are all tuning parameters, chosen by CV on the training data. (The figure uses the test set only to show the trade-off.)

Evaluating our candidate classifiers

This election's stations, scored against what the audit would find with no monitors:

Model	Test AUC	Share irregular, top 500
Logit	0.825	0.756
Pruned tree	0.821	0.736
Bagging	0.852	0.796
Random forest	0.858	0.818
Boosting	0.858	0.818
Truth	0.869	0.848

Remember, this table exists only because we simulated the data. A real audit comes **after** the monitors deploy. What does it see at the stations we covered? And what if malfeasance moves?

In R

```
f <- y ~ urban + no_agent + flagged + region + inc_share + turnout
fit_tree  <- rpart(f, data = train, method = "anova")   # leaf shares
fit_rf    <- ranger(update(f, factor(y) ~ .), data = train,
                    probability = TRUE, num.trees = 500, mtry = 2)
# X_tr: the six features as a numeric matrix
fit_boost <- xgboost(x = X_tr, y = factor(train$y), nrounds = 300,
                     learning_rate = 0.05, max_depth = 3)
```

We choose every tuning parameter in these calls by CV.

References

- James, G., Witten, D., Hastie, T. & Tibshirani, R. (2021) *An Introduction to Statistical Learning*, 2nd ed. – **8.1–8.2**. statlearning.com.
- Montgomery, J. M. & Olivella, S. (2018) "Tree-Based Models for Political Science Data." *American Journal of Political Science* 62(3).